



**TECNOLÓGICO
NACIONAL DE MÉXICO®**



INSTITUTO TECNOLÓGICO DE TEPIC

**INVESTIGACIÓN CUADRO
COMPARATIVO – PROGRAMACIÓN
DEL LADO DEL CLIENTE VS. DEL
LADO DEL SERVIDOR**

Ingeniería en Sistemas Computacionales

PROGRAMACIÓN WEB

Docnte:

Nava Hernández Irving Yair

Alumnos:

23400583 -Navarro Ramírez Gerson Asahel



1. Investiga y elabora un documento con los conceptos de programación del lado del cliente (client-side) y programación del lado del servidor (server-side).

Programación del Lado del Cliente (Client-Side / Front-End):

Es el desarrollo del código que se ejecuta directamente en el dispositivo del usuario (a través del navegador web). Su función principal es controlar la interfaz de usuario, el diseño visual y la interactividad en tiempo real (como animaciones, menús o validación de datos).

Entorno de Ejecución: La capacidad de procesamiento (CPU, RAM) la aporta la máquina del usuario final.

Interfaz y Experiencia de Usuario (UI/UX): Su objetivo primordial es gestionar la presentación visual, la disposición de elementos gráficos y la interactividad fluida en tiempo real.

Visibilidad del Código: Dado que se ejecuta localmente, el código fuente es público y totalmente legible a través de las herramientas de inspección del navegador.

Reducción de Latencia: Permite modificar datos en pantalla de forma instantánea sin necesidad de realizar peticiones constantes al servidor o recargar la ventana.

Programación del Lado del Servidor (Server-Side / Back-End):

Es el desarrollo del código que se ejecuta en una computadora remota (servidor) protegida. Su función principal es gestionar la lógica del negocio, la seguridad, la autenticación de usuarios y el procesamiento o almacenamiento de datos en bases de datos.

Lógica de Negocio y Reglas del Sistema: Controla los procesos internos de la aplicación (cálculos financieros, gestión de inventarios, algoritmos de recomendación, etc.).

Gestión de Datos y Persistencia: Administra el acceso directo a bases de datos para consultar, modificar o guardar información del sistema.

Seguridad e Aislamiento: El código fuente y las claves de acceso permanecen protegidas dentro del servidor. El usuario solo recibe el resultado procesado (por ejemplo, estructurado en JSON o HTML estático).

Control de Acceso e Identidad: Procesa la autenticación (logins) y autoriza qué información puede ver o modificar cada usuario según su rol.



2. Con base en la investigación, elabora un cuadro comparativo.

ASPECTO	CLIENT-SIDE	SERVER-SIDE
Ubicación de ejecución	Navegador web del usuario (dispositivo local: laptop, celular, tablet).	Servidor web remoto (computadoras en la nube o centros de datos).
Descripción general	Se enfoca en la capa visual, maquetación e interactividad directa con el usuario. El código se descarga y ejecuta localmente para manipular la interfaz sin refrescar la página.	Se enfoca en la lógica interna del negocio, almacenamiento, autenticación y procesamiento de datos. Procesa la información en segundo plano y devuelve un resultado estructurado.
Lenguajes más utilizados	JavaScript (lenguaje principal para la lógica), HTML (estructura) y CSS (estilos). (Nota: WebAssembly permite usar C++ o Rust en el navegador).	Python, Node.js (JavaScript), PHP, Java, C# (.NET), Ruby y Go.
Tecnologías y herramientas comunes	Frameworks/Librerías UI: React, Angular, Vue.js, Svelte. Emquetadores/Herramientas: Vite, Webpack, Tailwind CSS.	Frameworks: Express, Django, Laravel, Spring Boot, ASP.NET Core. Bases de Datos: PostgreSQL, MySQL, MongoDB, Redis.
Ventajas	• Respuestas inmediatas e interactividad fluida sin recargar pantalla. • Reduce la carga y peticiones directas al servidor. • Excelente experiencia de usuario en aplicaciones dinámicas (SPAs).	• Mayor seguridad: oculta la lógica de negocio y las credenciales. • Mejor rendimiento inicial y optimización para buscadores (SEO). • Funciona con independencia del rendimiento del dispositivo del usuario.
Desventajas	• La carga inicial de la página puede ser lenta si requiere bajar mucho código. • Dificultades con el SEO si no se configura bien el renderizado. • Depende de la potencia del dispositivo y tipo de navegador del cliente.	• Cada petición requiere comunicación con el servidor, generando posibles retardos en la UI. • Mayor costo de infraestructura y consumo de memoria/procesador en el servidor.
Seguridad	Baja respecto al código: El código es público y visible para cualquiera desde las herramientas del navegador. Propenso a ataques tipo XSS (Cross-Site Scripting) si no se desinfectan las entradas de	Alta respecto a la privacidad: El código fuente y la base de datos están completamente aislados del usuario. Previene vulnerabilidades como Inyección SQL o CSRF mediante validaciones estrictas



TECNOLÓGICO
NACIONAL DE MÉXICO®



	texto. Nunca se deben guardar claves o tokens privados aquí.	y control de acceso centralizado.
--	---	--------------------------------------